

BigQuery Incremental Sync Pipeline

MySQL → Google BigQuery | Architecture & Implementation

Form 26AS TDS Reconciliation System | Sundaram Finance Limited

Internship Project Report

Prepared by	Sanjana Ravishankar
Role	Full-Stack Developer Intern
Department	Data Platform
Organization	Sundaram Finance Limited
Technology Stack	Google Cloud Platform - BigQuery · Python SDK · MySQL 8.0 · Apache Beam
Project Period	May – July 2026
Report Date	July 2026

Table of Contents

1 Overview	3
2 Why Incremental – Not Truncate and Reload	3
3 Files	4
4 Watermark Strategy.....	5
4.1 MySQL Table - bq_sync_watermark.....	5
4.2 Watermark Per Table	5
5 Sync Strategy Per Table.....	5
5.1 income_records — MERGE via staging.....	5
5.2 form26as_records — WRITE_APPEND	6
5.3 cust_master — MERGE via staging	6
5.4 audit_logs — WRITE_APPEND	6
5.5 tds_summary — Full recompute + MERGE.....	6
6 Staging Tables — Why They Exist	7
7 Type Handling.....	7
8 How app.py BQ Functions and bq_incremental_sync.py Work Together	8
9 Verified Results	9
10 How to Run.....	10

1. Overview

This document covers the BigQuery incremental sync pipeline built for the Form 26AS TDS Reconciliation System at Sundaram Finance Limited. The pipeline replaced the original truncate-and-reload approach with a watermark-based incremental strategy that syncs only new rows on each run, satisfying the operations team's requirement for non-destructive, audit-safe data replication.

Pipeline at a Glance	
Source	MySQL 8 — form26as_db (operational database, source of truth)
Destination	Google BigQuery — project: eternal-skyline-468312-g7, dataset: tds_reconciliation
Strategy	Incremental — primary-key watermark, syncs new rows per run and updates any changes to the existing rows
Merge key	Primary key per table (income_id, record_id, cust_id, log_id)
Schedule	Nightly via Apache Airflow DAG (00:30 IST) or Windows Task Scheduler
Tables synced	income_records, form26as_records, cust_master, audit_logs, tds_summary
GCP location	asia-south1
Final Outcome	All 5 target tables verified with exact MySQL counts, zero duplicates

2. Why Incremental — Not Truncate and Reload

The original approach used WRITE_TRUNCATE — wiping and reloading every BQ table on each upload trigger. The operations team requested incremental load for three reasons:

- Audit logs must be append-only. Truncating them destroys the compliance trail which is a legal requirement for TDS reconciliation records.
- As data volume grows across Fleet, Oracle Finance, and Leasing business units, full reloads become expensive in BigQuery slot usage and time.
- Incremental load enables a nightly scheduled sync independent of user activity — Power BI dashboards are always fresh in the morning regardless of when uploads happened.

i CDC (Change Data Capture via Debezium + Kafka) was evaluated but not adopted. At Sundaram Finance's current transaction volume, a nightly Airflow cron achieves the same outcome with far simpler infrastructure. CDC is recommended only if real-time Power BI refresh becomes a requirement in future.

3. Files

File	Purpose
bq_incremental_sync.py	Core incremental sync engine. Watermark-based PK strategy. Runs all 5 tables. Called via python bq_incremental_sync.py
create_bq_tables.py	Creates all BQ tables and 3 staging tables. Run once after schema changes. Includes tds_deductible field and all staging tables.
dataflow_mysql_to_bq.py	Apache Beam pipeline for GCP Dataflow deployment. For future large-scale cloud execution. Fixed: tds_deductible added, WRITE_APPEND set.

4. Watermark Strategy

The operational MySQL tables do not have consistent timestamp columns across all tables. To avoid altering production tables (which were already submitted and reviewed), the pipeline uses the table's auto-increment primary key as the watermark instead of a created_at timestamp.

This is stored in a dedicated MySQL table called bq_sync_watermark, created automatically by the sync script on first run.

4.1 MySQL Table - bq_sync_watermark

```
CREATE TABLE IF NOT EXISTS bq_sync_watermark (
  table_name      VARCHAR(100) NOT NULL PRIMARY KEY,
  last_synced_at  DATETIME     NOT NULL DEFAULT '2000-01-01 00:00:00',
  last_row_count  INT          NOT NULL DEFAULT 0,    -- stores last synced PK
  updated_at      DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP
                ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

One row per synced table. last_row_count stores the last synced primary key value. last_synced_at stores the timestamp of the sync run for auditing. updated_at is auto-maintained by MySQL.

4.2 Watermark Per Table

Table Watermark Strategy	
income_records	income_id (auto-increment PK) — no created_at in live table. New uploads always generate higher income_ids so all new rows are captured.
form26as_records	record_id (auto-increment PK) — no timestamp column in live table. Records never updated after insert so PK watermark is sufficient.
cust_master	cust_id (auto-increment PK) — created_at exists in schema but not in live table. New customers always get higher cust_ids.
audit_logs	log_id (auto-increment PK) — timestamp exists but named "timestamp" not "created_at". Audit logs are append-only, PK watermark is correct.
tds_summary	No watermark — always full recompute across all FYs. Derived table; one new invoice changes the aggregate for the entire FY.

i For income_records and form26as_records, the last_row_count field stores the last synced PK value (not an actual row count). The field name is reused since no timestamp-based watermark was available without altering production tables.

5. Sync Strategy Per Table

5.1 income_records — MERGE via staging

Uses a two-step staging + MERGE pattern. Handles both new rows AND updates to tds_deductible which changes when update_tds_deductible_from_section() runs after a Form 26AS upload.

- Step 1: Fetch rows from MySQL WHERE income_id > last_synced_id
- Step 2: Load to income_records_staging in BQ — WRITE_TRUNCATE (clears stale staging data only)
- Step 3: BigQuery MERGE on income_id — UPDATE matched rows, INSERT new rows
- Step 4: Store MAX(income_id) as new watermark in bq_sync_watermark

Note: No MERGE via staging for all rows. Instead, new rows → APPEND, updated rows (tds_deductible delta) → staging → MERGE only the delta.

i Explicit BQ schema provided for staging (autodetect=False) — prevents BigQuery from autodetecting doc_date as DATE type. The main table stores doc_date as STRING; a type mismatch causes the MERGE SQL to fail at runtime.

5.2 form26as_records — WRITE_APPEND

Form 26AS records are never updated after insert — pure append-only. Simple incremental append using record_id as watermark. No staging or MERGE needed.

- Fetch rows WHERE record_id > last_synced_id
- Append directly to BQ form26as_records using insert_rows_json in batches of 500
- Store MAX(record_id) as new watermark only if errors = 0

5.3 cust_master — MERGE via staging

Same staging + MERGE pattern as income_records. MERGE on cust_id handles both new customers and re-uploaded customer data correctly. Customer changes made via the app also trigger an immediate bq_sync_cust_master() in app.py — the nightly job catches anything remaining.

Note: New customers → APPEND, changed customers → staging → MERGE delta only.

5.4 audit_logs — WRITE_APPEND

Audit logs are purely append-only. Uses log_id as watermark. The BQ audit_logs table required an ALTER TABLE to add severity_level, user_name, and timestamp columns before the incremental sync could populate it — the table was originally created with a different schema.

5.5 tds_summary — Full recompute + MERGE

Recomputed across all financial years on every run using SUM(tds_deductible) — not tax_amt. MERGE on (cust_id, fin_year) prevents duplicates. Cannot be incrementally synced because one new invoice changes the aggregate for the entire FY.

Note: New (cust_id, fin_year) → APPEND, changed aggregates → staging → MERGE delta only.

6. Staging Tables — Why They Exist

BigQuery does not support direct row-level UPDATE via the Python SDK. To update existing rows (for example when `tds_deductible` changes after recalculation), a MERGE SQL statement is required. MERGE needs two tables — a source and a target. The staging table is the source.

The staging table is wiped and reloaded on every sync run — `WRITE_TRUNCATE` on staging is intentional and safe because the staging table is transient. The main tables are never truncated after initial setup.

However, the staging table only receives the genuinely changed rows identified by comparison against BQ.

Staging Tables	
income_records_staging	Holds changed <code>income_records</code> rows. Explicit schema forces <code>doc_date</code> as <code>STRING</code> to match main table.
cust_master_staging	Holds new or changed customer rows. MERGE on <code>cust_id</code> .
tds_summary_staging	Holds recomputed TDS summary rows. MERGE on <code>(cust_id, fin_year)</code> .

7. Type Handling

MySQL returns Python-typed objects that Python's `json` module cannot serialise by default. Two layers of type handling are applied before rows reach BigQuery:

Layer 1 — `clean_rows()`

Applied to all rows fetched from MySQL. Converts every value to a BQ-safe Python type.

Type Conversion	
MySQL DATE	Python date object → string 'YYYY-MM-DD' via <code>strftime</code> . date checked separately from <code>datetime</code> (date has no <code>.hour</code> attribute).
MySQL DATETIME	Python datetime object → string 'YYYY-MM-DD HH:MM:SS' via <code>strftime</code>
MySQL DECIMAL	Python Decimal object → Python float
MySQL BOOL	Python bool → bool. Checked before int — bool is a subclass of int in Python; checking int first would silently convert <code>True</code> →1 and <code>False</code> →0.
MySQL INT	Python int → int (no conversion)
MySQL VARCHAR	Python str → str (no conversion)
NULL	Python None → None (BigQuery NULL)

Layer 2 — `_serial()` for staging loads

`load_table_from_json()` calls `json.dumps` internally. A custom JSON serialiser is applied before every staging load as a second safety layer:

```
def _serial(o):
    if isinstance(o, datetime.datetime): return o.strftime('%Y-%m-%d %H:%M:%S')
    if isinstance(o, datetime.date):     return o.strftime('%Y-%m-%d')
    if isinstance(o, decimal.Decimal):   return float(o)
    raise TypeError(f'Not serialisable: {type(o)}')

safe_rows = json.loads(json.dumps(rows, default=_serial))
```

Applied to all three staging loads. Not needed for `bq_append()` since `insert_rows_json` handles plain Python types natively.

8. How app.py BQ Functions and bq_incremental_sync.py Work Together

Two sync mechanisms complement each other and together ensure BQ is never more than one night stale:

Key Metrics	app.py bq_sync_* functions	bq_incremental_sync.py
When	Immediately after each upload	Nightly on schedule
What	Only the FY just uploaded	All tables, all FYs
Trigger	Automatic — fired by Flask upload handlers	Manual or Airflow / Task Scheduler cron
Strategy	WRITE_APPEND via insert_rows_json	MERGE + APPEND with PK watermark
Purpose	Real-time — BQ updated within seconds of upload	Safety net — comprehensive nightly refresh of all tables

9. Verified Results

Full pipeline tested thoroughly across multiple sessions. Final clean state verified on 22nd July.

1. Final Sync Output

- ✓ income_records: merged 51 rows, watermark → income_id 722 (errors: 0)
- ✓ form26as_records: appended 41 rows (errors: 0)
- ✓ cust_master: merged 15 rows (errors: 0)
- ✓ audit_logs: appended 39 rows (errors: 0)
- ✓ tds_summary: merged 15 rows across 3 FYs (errors: 0)

2. Final BigQuery Row Counts — Exact Match with MySQL

BigQuery Verified Counts	
income_records	51 rows ✓ — exact match with MySQL income_records
form26as_records	41 rows ✓ — exact match with MySQL form26as_records
cust_master	15 rows ✓ — exact match with MySQL cust_master
audit_logs	39 rows ✓ — exact match with MySQL audit_logs
tds_summary	15 rows ✓ — exact match with MySQL aggregate (3 FYs × 5 customers average)

3. Reconciliation Accuracy Verified in BigQuery

LEENA (Section 194A, FY 2020-21) verified directly in BigQuery tds_summary:

LEENA — BigQuery Verification	
fin_year	FY 2020-21
total_tds_deductible	Rs 5,070.00
total_26as_credit	Rs 5,070.00
variance_amt	Rs 0.00
Result	Perfectly reconciled — TDS computation matches Form 26AS credit exactly

✓ BigQuery data matches MySQL source of truth — incremental sync is accurate with zero variance

Calculation trace: Section 194A, rate 10%, threshold Rs 40,000. Annual taxable Rs 50,700 > threshold → TDS applies. 3 invoices × 10% = Rs 5,070. Form 26AS credit = Rs 5,070. Variance = Rs 0.00.

10. How to Run

First-time setup (completed — for reference only)

- Step 1: python create_bq_tables.py — creates all BQ tables including 3 staging tables
- Step 2: python bq_incremental_sync.py — first incremental run, seeds BQ with all existing MySQL data and sets all watermarks

Manual run anytime

```
python bq_incremental_sync.py
```

Idempotent — running multiple times will not create duplicates. On second run with no new MySQL data, all tables report "no new rows" and exit cleanly in seconds.

Troubleshooting

Issue → Fix	
errors > 0 on audit_logs	Run ALTER TABLE in BQ to add missing columns (severity_level, user_name, timestamp). Reset watermark: UPDATE bq_sync_watermark SET last_row_count=0 WHERE table_name='audit_logs'. Re-run.
DATE/STRING type mismatch	Staging schema uses explicit schema (autodetect=False) with doc_date as STRING. Already fixed in current bq_incremental_sync.py.
ModuleNotFoundError: dotenv	pip install python-dotenv inside beam_env
BQ table not found	Run python create_bq_tables.py to create missing tables including staging tables.